

Introduction

HET itself is explained in the README file. This document explains some of the design choices in the C implementation of the language.

Walkthrough

The C sources are divided up into 2 files: het.c and gc.c. The file het.c contains the main body of the code. It is logically subdivided into several unnamed sections. These sections could have been separate files, but because the total number of lines is so small, they are more easily seen in one file.

Technicalities

The empty list, `()`, is internally represented with an intptr_t that has all bits set to zero, or as a pointer to a list with no entries. The two are treated as equal by `=`.

Specials `+` and `/` destroy the list they are working on; `!`, `:`, and `%` make a shallow copy. Specials `.` and `;` can operate on an empty WS; all other specials require at least one item on the WS.

Behaviour that is not expected is captured with an assert statement unless the code has been compiled with -DNDEBUG. Such behaviour should therefore be considered as undefined. For all such behaviour: the source code will be the ultimate reference.

Limitations

At the start of het.c there is a define MAX_IDENT that restricts the number of unique characters in a name to 100. It doesn't seem necessary to have an unlimited number of characters in a name and using a static array is the easiest implementation but comes with a maximum number that must be set to a certain value and 100 seems generous enough. In reality I wouldn't want names longer than 30 characters.

Another limitation is contained in markfactor that calls marklist that calls markfactor. This can't go on forever. There is a recursion limit imposed by compiler and operating system. If data structures become convoluted enough

that this limit becomes a problem, then a technique called pointer reversal can help to do the marking phase without using the recursion stack. At this moment I don't see any deep data structures that warrant such an implementation.

Another limitation is in the character set that can be used to build a word. This is currently limited to alphanumerics and the underscore character. If it becomes necessary to write floating point literals, then some more characters should be allowed in a name, but at this moment no floating point literals are needed.

When tracing the execution of a program, only one next symbol is shown. It is possible to show all of the future, but then the display becomes a little messy, so the future is now restricted to only the next symbol.

Future plans

Now that the future has been mentioned, what improvements can be made to het.c? Lifting some of the limitations is one improvement. Compiling the code could be another. Compiling could be used to get rid of the symbol tables.

The nice thing about how HET executes, is that it doesn't use stack space, unless a call to `gc` is inserted in HET source code. HET does use allocated memory and that makes it not eligible for embedded systems.